

BitFS 详细设计

本文档为 BitFS 设计文档体系的第三层：算法、数据结构、协议细节。

文档体系：(总体设计) 1. 概念设计 — 项目愿景、核心概念、架构概览 2. 系统设计 — 模块划分、接口定义、数据流 3. 详细设计（本文档）— 算法、数据结构、协议细节 4. 测试用例 — 测试用例设计

每个 B 节对应系统设计中同编号章节的详细展开。

B 节与系统设计章节对应关系：

详细设计 B 节	对应系统设计章节	说明
二-B HD 钱包	二 HD 钱包	直接对应
四-B Metanet 交易	四 Metanet 交易格式	直接对应
五-B Koblitz 加密	五 数据类型与加密模型	直接对应
六-B Rabin 签名	十一-b Rabin 签名验证	编号偏移
七-B CLTV 时间锁	十一-c CLTV 时间购买	编号偏移
八-B 内容分片	十一-d 大文件分片	编号偏移
九-B CLI 命令	九 bitfs 命令	直接对应
十一-B Token	十一 买卖交易	直接对应
十三-B 通信协议	十三 Daemon	直接对应
十四-B 收益权 ISO	十一-f 收益权代币化	编号偏移
十五-B BIP32 访问控制	(跨章节)	系统设计无独立章节
十六-B Paymail	六 DNSLink / Paymail	编号偏移

二-B、HD 钱包派生规则详细设计

本节补充 HD 钱包的精确派生路径、Method 42 密钥派生公式和钱包存储格式，与 `src/internal/method42/hdwallet.go` 和 `encrypt.go` 的实现一致。

A. BIP39 助记词

熵长度：

12 个单词 → 128 bits 熵 (Mnemonic12Words)

24 个单词 → 256 bits 熵 (Mnemonic24Words)

生成流程：

1. `entropy = random(bitSize)`

2. `mnemonic = bip39.NewMnemonic(entropy)`

Passphrase (可选)：

`seed = bip39.NewSeed(mnemonic, passphrase)`

→ `PBKDF2(mnemonic_bytes, "mnemonic" + passphrase, 2048, 64, SHA512)`

`passphrase` 为空字符串时仍参与派生 (""≠无)

Master Key：

`master = bip32.NewMaster(seed, network_params)`

→ `HMAC-SHA512(key="Bitcoin seed", data=seed)`

→ 左 32 字节 = 私钥，右 32 字节 = chain code

B. BIP32 HD 密钥树完整路径表

BIP44 常量 (`hdwallet.go`):

`PurposeBIP44 = 44`

`CoinType = 236` (BitFS 注册的 BIP44 coin type)

`FeeAccount = 0` (费用密钥链 account index)

`DefaultVaultAccount = 1` (第一个 Vault 的 account index)

`ExternalChain = 0` (接收地址链)

`InternalChain = 1` (找零地址链)

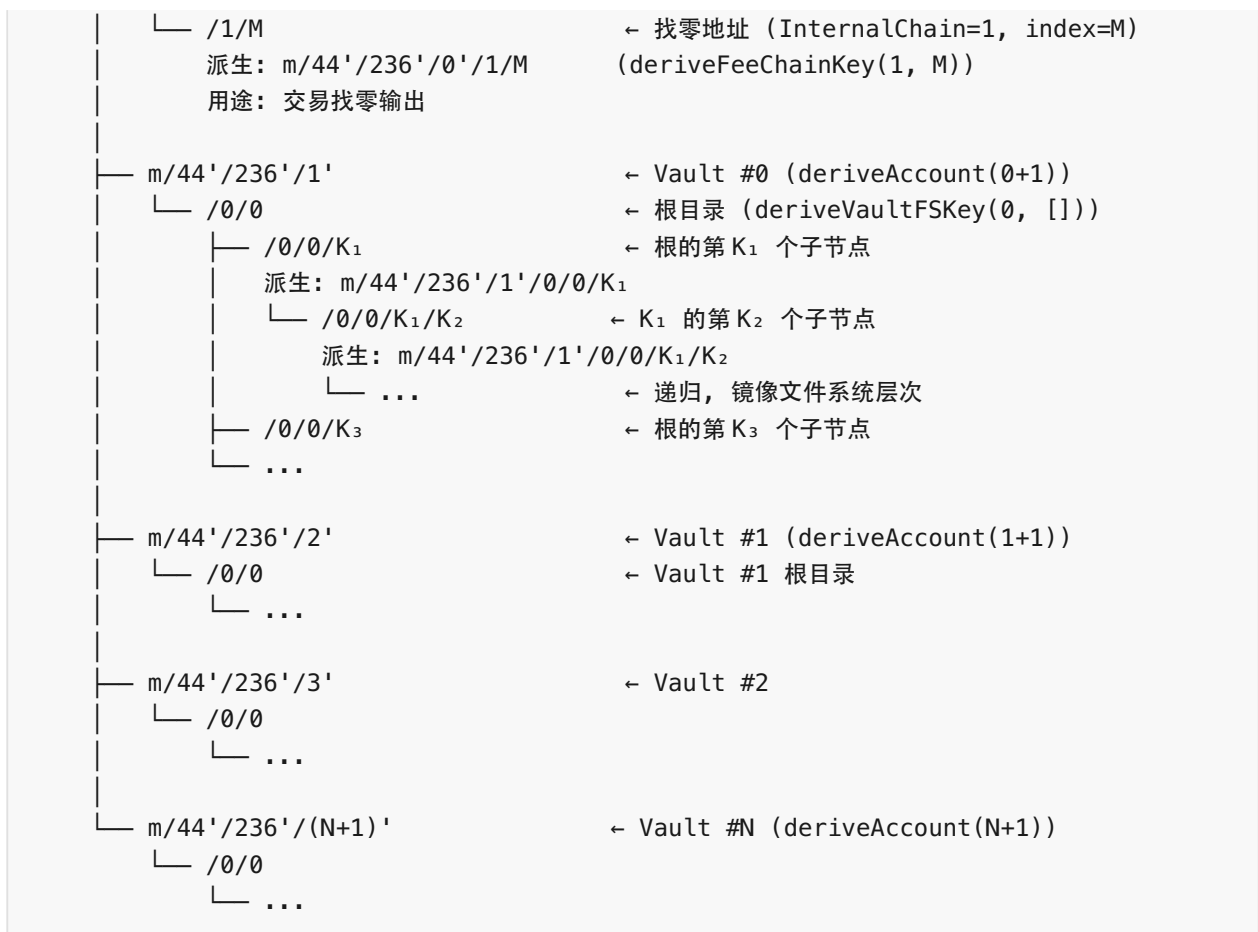
`MaxFileIndex = 2^31-1` (BIP32 非硬化子密钥上限)

`MaxPathDepth = 64` (文件系统最大嵌套深度，超过此深度返回错误)

完整密钥树

HD Seed (BIP39 助记词 + 可选 passphrase)

```
|
|├── m/44'/236'/0'          ← 费用密钥链 (deriveAccount(0))
|│   ├── /0/M              ← 接收地址 (ExternalChain=0, index=M)
|│   │   派生: m/44'/236'/0'/0/M    (deriveFeeChainKey(0, M))
|│   │   用途: 充值地址 (feeReceiveAddress)
```



派生函数映射

函数	路径	用途
deriveAccount(account)	m/44' /236' /account'	BIP44 account 层
deriveFeeChainKey(chain, index)	m/44' /236' /0' /chain/index	费用密钥
deriveVaultFSKey(vaultIndex, filePath)	m/44' /236' / (vaultIndex+1)' /0/0[/filePath...]	文件系统节点密钥
DeriveNodeKey(vault, filePath)	同上	导出 (pub, priv)
DeriveNodePubKey(vault, filePath)	同上	仅导出 pub
DeriveFeeKey(chain, index)	m/44' /236' /0' /chain/index	导出 (pub, priv)

C. 文件系统路径 → HD 路径映射规则

Index 分配

每个目录维护 next_child_index (uint32, 从 1 开始单调递增):

创建子节点时:

```
file_index = parent.Payload.NextChildIndex
```

若 `file_index == 0` → 回退: `len(parent.Children) + 1`

创建后: `parent.NextChildIndex = file_index + 1`

特殊情况:

根目录创建时: `next_child_index = 1` (初始值)

首个子节点: `index = 1`

第二个子节点: `index = 2`

以此类推...

规则

规则	说明
删除后不复用	rm/rmdir 不减少 next_child_index, 被删除的 index 永久占用
mv 同目录	不改变 index (仅改名)
mv 跨目录	目标节点获得新 index (来自目标目录的 next_child_index)
cp	目标节点获得新 index + 新 HD 路径
硬链接	消耗 next_child_index 但不创建新 HD 密钥 (ChildEntry 复用已有 P_node)
软链接	消耗 next_child_index 并创建新 HD 密钥 (LINK 节点是新节点)

HD 路径构建

```
// deriveVaultFSKey 构建完整路径
func deriveVaultFSKey(vaultIndex uint32, filePath []uint32):
    account = deriveAccount(vaultIndex + DefaultVaultAccount) // m/44'/236'/(N+1)'
    chain  = account.Child(0)                                // /0
    root   = chain.Child(0)                                  // /0/0

    current = root
```

```

for _, idx := range filePath:
    current = current.Child(idx)           // /0/0/K1 /K2 /...

return current

// parentFilePath 递归构建从根到该节点的完整路径
func parentFilePath(node MetanetNode) []uint32 {
    if node.Payload.Index == 0 {
        return []uint32{} // 根节点: 空路径 (对应 m/44'/236'/V'/0/0)
    }
    parent := LookupParentNode(node) // 通过 Metanet 父边查找
    return append(parentFilePath(parent), node.Payload.Index)
}

// 完整 HD 路径构建 (含硬化/非硬化标记)
func fullHDPATH(vault_index uint32, node MetanetNode) string {
    base := fmt.Sprintf("m/44'/236'/%d'/0/0", vault_index)
    for _, idx := range parentFilePath(node) {
        if node.ChildEntry.Hardened {
            base += fmt.Sprintf("/%d'", idx)
        } else {
            base += fmt.Sprintf("/%d", idx)
        }
    }
    return base
}

// 创建子节点时:
filePath = append(parentFilePath(parentNode), fileIndex)
P_node, D_node = wallet.DeriveNodeKey(vault, filePath)

```

性能说明: 深层路径需遍历完整祖先链。实现时应缓存 node→path 映射。

重要限制: $\text{MaxFileIndex} = 2^{31} - 1$ (2147483647)。超过此值的 `file_index` 将被拒绝 (`ErrFileIndexOutOfRange`)。这是 BIP32 非硬化派生的限制。

深度限制: $\text{MaxPathDepth} = 64$ 。64 层嵌套覆盖所有实际使用场景, 避免极深路径的 HMAC-SHA512 串行计算开销。超过此深度的 `put` 操作将返回 `ErrPathTooDeep` 错误。

D. Method 42 加密密钥派生

加密密钥派生公式 (新方案, 保留 BIP32 代数关系)

三种 access 模式:

FREE:

`D_node` 使用标量 1

```
aes_key = KDF(ECDH(1, P_node), key_hash) = KDF(P_node, key_hash)
→ P_node 通过 DNSLink 公开, 任何人可计算
```

PRIVATE / PAID:

```
aes_key = KDF(ECDH(D_node, P_node), key_hash)
→ 仅知道 D_node 的 Owner 或通过 HTLC 获得 capsule 的 Buyer 可计算
```

完整流程:

```
1. key_hash = SHA256(SHA256(plaintext))      ← ComputeKeyHash
2. S_node = ECDH(D_node, P_node)             ← BIP32 密钥直接参与 ECDH
   → FREE: S_node = ECDH(1, P_node) = P_node
   → PRIVATE/PAID: S_node = D_node × P_node
3. aes_key = HKDF-SHA256(
    ikm = S_node.x,                          // 共享密钥 x 坐标 (32 bytes)
    salt = key_hash,                         // 内容承诺 + 唯一性
    info = "bitfs-file-encryption",
    length = 32                             // AES-256 密钥
)
```

关键: BIP32 代数关系保留 - $S_{child} = S_{parent} + offset \times P_{buyer}$,
使得目录树级 capsule 派生可行 (详见 十五-B)。

加密格式

加密 (aesGCMEncrypt):

```
nonce = random(12 bytes)                    ← NIST SP 800-38D
output = nonce || AES-256-GCM(plaintext, aes_key, nonce) || tag
→ nonce(12B) + ciphertext + GCM_tag(16B)
```

内容存储: daemon 以 key_hash 为键存储加密数据 (内部实现细节)

解密 (aesGCMDecrypt):

```
nonce = data[0:12]
ciphertext = data[12:]                     ← 包含 GCM tag
plaintext = AES-256-GCM.Open(ciphertext, aes_key, nonce)
```

Nonce 安全分析: 12 字节随机 nonce 在同一密钥下 2^{48} 次加密后碰撞概率达 50%。BitFS 中不同文件使用不同 aes_key (由 ECDH 派生), 跨文件不存在 nonce 碰撞风险。同一文件重复加密次数远低于 2^{48} , 故随机 nonce 方案安全。

PRIVATE 模式 Payload Envelope

加密 (EncryptPrivatePayload):

```
1. data = proto.Marshal(完整 BitFSPayload)
2. key_hash = SHA256(SHA256(data))          ← 注: 以序列化 payload 为输入
3. file_index = payload.Index
```

```

4. S_node = ECDH(D_node, P_node).x
5. aes_key = HKDF-SHA256(ikm=S_node, salt=key_hash, info="bitfs-file-encryption")
6. enc_payload = nonce(12B) || AES-256-GCM(data, aes_key) || tag

```

返回 envelope:

```

BitFSPayload {
  encrypted: true,
  private_key_hash: key_hash (32B, 明文),
  private_file_index: file_index (明文),
  enc_payload: enc_payload
}
→ 其余字段为零值

```

解密 (DecryptPrivatePayload):

```

1. key_hash = envelope.private_key_hash
2. file_index = envelope.private_file_index
3. S_node = ECDH(D_node, P_node).x
4. aes_key = HKDF-SHA256(ikm=S_node, salt=key_hash, info="bitfs-file-encryption")
5. data = AES-256-GCM.Open(enc_payload, aes_key)
6. payload = proto.Unmarshal(data)

```

恢复可行性:

- D_node: 从 HD 种子 + 文件路径确定性派生
- private_key_hash + private_file_index: 在 envelope 明文中
- 即使丢失其他所有数据, 仅凭助记词 + envelope 即可恢复

模式转换 (ReEncrypt)

ReEncrypt(encryptedData, D_node, P_node, key_hash, file_index, fromAccess, toAccess):

```

1. plaintext = Decrypt(encryptedData, D_node, P_node, key_hash, file_index, fromAccess)
2. result = Encrypt(plaintext, D_node, P_node, file_index, toAccess)
→ 新 ciphertext (新随机 nonce), 新 key_hash

```

支持的转换:

```

FREE → PRIVATE (encrypt 命令)
PRIVATE → FREE (decrypt 命令)

```

E. 钱包存储格式

WalletState 结构

```

{
  "encrypted_seed": "<base64>", // AES-256-GCM 加密的 BIP39 seed
  "mnemonic": "", // 初始化后清空 (仅首次显示)
  "next_receive_index": 0, // 费用链下一个接收地址索引
}

```

```

"next_change_index": 0,          // 费用链下一个找零地址索引
"utxos": [                      // 跟踪的 UTXO 集合
{
    "txid": "<32 bytes hex>",
    "vout": 0,
    "amount": 10000,
    "spent": false,
    "chain": 0,                  // 0=external, 1=internal
    "address_index": 0
}
]
}

```

Seed 加密方案

```

// 钱包加密（首次创建或修改密码时）
salt = crypto.RandomBytes(16)          // 每次加密生成新 salt
derived_key = Argon2id(password, salt, params{
    Time:      3,                // 迭代次数
    Memory:    65536,           // 64 MB 内存
    Parallelism: 4,             // 4 线程
    KeyLen:    32,              // 256-bit 密钥
})

// wallet.enc 文件格式: salt(16B) || nonce(12B) || ciphertext
nonce = crypto.RandomBytes(12)
ciphertext = AES-256-GCM.Encrypt(derived_key, nonce, seed || checksum)
WriteFile("wallet.enc", salt || nonce || ciphertext)

```

安全说明: 单次 SHA256 可被 GPU 以每秒数十亿次的速度暴力破解。Argon2id 通过 要求大量内存 (64MB) 和多次迭代, 将暴力破解成本提升数个数量级。此密钥保护整个 HD 钱包种子 (所有文件加密密钥的根), 是系统最关键的安全环节。

```

// 钱包解锁
data = ReadFile("wallet.enc")
salt = data[0:16]
nonce = data[16:28]
ciphertext = data[28:]
derived_key = Argon2id(password, salt, same_params)
seed || checksum = AES-256-GCM.Decrypt(derived_key, nonce, ciphertext)

```

持久化

```

存储键: "wallet_state"
格式: JSON (json.Marshal/Unmarshal)

```


接口: WalletStore { Get(key), Set(key, value), Delete(key), Close() }

UTXO 索引跟踪:

- AddUTXO: 追加到 UTXOs 列表
 - 若为 external 链且 $index == next_receive_index$ → 自动递增
 - 若为 internal 链且 $index == next_change_index$ → 自动递增
- SpendUTXO: 按 txid+vout 查找并标记 spent=true
- SelectUTXOs: 贪心选择未花费 UTXO 直到满足目标金额

F. 恢复流程

助记词恢复

bitfs wallet restore:

1. 输入 mnemonic (12 或 24 个单词)
2. 验证: `bip39.IsMnemonicValid(mnemonic)`
3. 输入可选 passphrase
4. `seed = bip39.NewSeed(mnemonic, passphrase)`
5. `master = bip32.NewMaster(seed, network)`
6. 加密 seed → `encrypted_seed`
7. 初始化 WalletState (UTXOs 为空, indexes 为 0)

恢复后可派生:

- 所有 fee keychain 地址 (`m/44'/236'/0'/chain/index`)
- 所有 Vault 根密钥 (`m/44'/236'/(N+1)'/0/0`)
- 所有文件系统节点密钥 (按已知路径)

交易数据恢复

助记词恢复 仅 恢复密钥树。以下数据需从备份恢复:

- `~/.bitfs/txstore/` 交易数据 + Merkle proof
- `~/.bitfs/headers/` 区块头链
- `~/.bitfs/daemon.db` Daemon 状态

备份建议: 用户自行备份 `~/.bitfs/` 目录

密钥恢复的确定性保证

给定相同的 (mnemonic, passphrase):

- 相同的 seed
- 相同的 master key
- 相同的所有派生密钥

给定相同的 (D_node, P_node, key_hash):

- 相同的 `S_node = ECDH(D_node, P_node).x`
- 相同的 `aes_key = HKDF-SHA256(ikm=S_node, salt=key_hash, info="bitfs-file-`

```
encryption")
```

→ 可解密对应文件

给定 PRIVATE envelope (encrypted=true, private_key_hash, private_file_index):

→ D_node 从路径派生, P_node 从 D_node 计算

→ aes_key = HKDF-SHA256(ikm=ECDH(D_node, P_node).x, salt=private_key_hash, info="bitfs-file-encryption")

→ 可解密 enc_payload → 恢复完整 Protobuf 元数据

G. 网络配置

种子级别网络绑定: `bitfs init --network <network>` 时选定, 所有 Vault 共享同一网络。多网络需求通过 `BITFS_HOME` 环境变量隔离。

网络参数结构

```
// NetworkConfig 定义网络参数
type NetworkConfig struct {
    Name      string `json:"name"          // 网络标识
    AddressVersion byte  `json:"address_version" // P2PKH 地址前缀 (mainnet: 0x00, testnet: 0x6f)
    P2SHVersion byte  `json:"p2sh_version"    // P2SH 地址前缀 (mainnet: 0x05, testnet: 0xc4)
    DefaultPort uint16 `json:"default_port"    // P2P 端口
    RPCPort     uint16 `json:"rpc_port"        // RPC 端口
    DNSSeeds    []string `json:"seeds"           // 种子节点
    GenesisHash string `json:"genesis_hash"    // 创世块哈希 (hex)
}
```

预设网络参数

```
var (
    MainNet = NetworkConfig{
        Name:      "mainnet",
        AddressVersion: 0x00,
        P2SHVersion: 0x05,
        DefaultPort: 8333,
        RPCPort:     8332,
        DNSSeeds:    []string{"seed.bitcoinsv.io", "seed.satoshisvision.network"},
        GenesisHash: "000000000019d6689c085ae165831e934ff763ae46a2a6c172b3f1b60a8ce26f",
    }
    TestNet = NetworkConfig{
        Name:      "testnet",
        AddressVersion: 0x6f,
        P2SHVersion: 0xc4,
        DefaultPort: 18333,
        RPCPort:     18332,
        DNSSeeds:    []string{"testnet-seed.bitcoinsv.io"},
        GenesisHash: "000000000933ea01ad0ee984209779baaec3ced90fa3f408719526f8d77f4943",
    }
)
```

```

}
// Teranode 测试网 (实验性, 参数待 BSV 官方确认后补齐)
TeraTestNet = NetworkConfig{
    Name:      "teratestnet",
    AddressVersion: 0x6f,
    P2SHVersion: 0xc4,
    DefaultPort: 0, // 待 BSV 官方确认
    RPCPort:    0, // 待 BSV 官方确认
    DNSSeeds:   []string{},
    GenesisHash: "", // 待 BSV 官方确认
}
RegTest = NetworkConfig{
    Name:      "regtest",
    AddressVersion: 0x6f,
    P2SHVersion: 0xc4,
    DefaultPort: 18444,
    RPCPort:    18443,
    DNSSeeds:   nil, // 无种子节点, 使用 node_rpc
    GenesisHash: "0f9188f13cb7b2c71f2a335e3a4fc328bf5beeb436012afca590b1a11466e2206",
}
)

// GetNetwork 根据名称获取预设网络, 不存在则尝试加载自定义配置
func GetNetwork(name string) (*NetworkConfig, error)

```

本地存储结构

```

~/bitfs/
├─ wallet.enc           # AES-256-GCM(seed, passphrase_key)
├─ network.json         # NetworkConfig (预设名或完整自定义配置)
├─ vaults.json          # [{name, account_index, root_txid}]
├─ keys/               # 密钥缓存 (购买的 capsule)
├─ spv/                # SPV 数据
│   └─ headers.db       # header chain
│   └─ txstore.db       # 交易存储
└─ storage/            # 内容寻址存储

```

network.json 格式

预设网络 (简写):

```
{"name": "mainnet"}
```

自定义网络 (完整):

```
{
  "name": "my-private-net",
  "address_version": "0x6f",

```

```

"p2sh_version": "0xc4",
"default_port": 19333,
"rpc_port": 19443,
"seeds": ["node1.company.internal:19333"],
"genesis_hash": "0x00000000..."
}

```

网络对密钥派生的影响

HD 路径不因网络而变 (`m/44'/236'/N'` 始终相同)。网络影响:

组件	影响
地址编码	version byte 不同 → 地址字符串不同
交易广播	广播到不同 P2P 网络
SPV 验证	不同的 header chain
费率策略	mainnet 市场费率, testnet/regtest 最低费率
Daemon 发现	DNS SRV (mainnet/testnet) vs localhost (regtest)

同一 HD 种子在不同网络生成相同密钥对, 但地址不同 (version byte), 交易不互通 (不同链)。种子级别隔离确保不会误操作。

四-B、Metanet 交易
结构详细设计

五-B、Koblitz 加密详细设计

本节补充 Koblitz 曲线加密 (secp256k1 ECC) 与 AES-256-GCM 混合加密的算法细节。与 Bitcoin 使用同一椭圆曲线体系。

A. 混合加密架构

`` 加密流程: 1. 生成对称密钥: $S_k = \text{random}(32 \text{ bytes})$ 2. Koblitz 加密 S_k : → 将 S_k 视为椭圆曲线上的点映射输入 → 使用接收方公钥

四-B、Metanet 交易结构详细设计
P_recipient 进行 ECC 加密 → encrypted_S_k ≈ 2KB (32 字节 → ~64 个 ECC 点, 每点 33 字节) 3. AES-256-GCM 加密内容: → nonce = random(12 bytes) → ciphertext = AES-256-GCM(plaintext, S_k, nonce) 4. 输出: encrypted_S_k nonce(12B) ciphertext GCM_tag(16B)
解密流程: 1. 解析: encrypted_S_k, nonce, ciphertext 2. Koblitz 解密: → 使用接收方私钥 D_recipient 解密 encrypted_S_k → S_k 3. AES-256-GCM 解密: → plaintext = AES-GCM.Open(ciphertext, S_k, nonce) ``
B. 为什么是 Koblitz + AES 混合
`` Koblitz (secp256k1) 加密特性: - 逐字符加密: 每字节映射到椭圆曲线上的点 - 膨胀率: 1:33 至 1:66 (每字节 → 一个压缩/非压缩 ECC 点) - 适合: 小数据 (密钥, 哈希值, 短消息) - 不适合: 大文件

四-B、Metanet 交易结构详细设计
(100KB 文件 → 3-6MB 密文)
AES-256-GCM 加密特性: - 对称加密: 固定膨胀 (nonce 12B + tag 16B) - - 适合: 任意大小数据 - 需要: 安全的密钥分发
混合方案: - Koblitz 加密 32 字节对称密钥 → ~2KB 开销 (可接受) - AES-256-GCM 加密 bulk 内容 → 固定 28B 开销 (高效) - 与 Bitcoin 同密码体系 (secp256k1), 复用现有密钥基础设施 ``
C. Koblitz 点映射算法
`` 字节 → 椭圆曲线点 (Koblitz mapping):
对于每个字节 b (0-255): 1. 构造候选 $x = b * k + j$ ($j = 0, 1, 2, \dots$) 其中 k 是足够大的常数确保不同字节映射不冲突 2. 检查 $x^3 + 7 \pmod{p}$ 是否为二次剩余 (secp256k1 曲线方程: $y^2 = x^3 + 7$) 3. 若是 → $P_b = (x, y)$ 是曲线上的点 4. 若否 → $j++$, 重试
加密点 P_b: 1. 选择随机 r 2. $C1 = r \times G$ (ephemeral 公钥) 3. $C2 = P_b + r \times P_{recipient}$

四-B、Metanet 交易结构详细设计
(加密后的点) 4. 密文 = (C1, C2)
解密: 1. $P_b = C2 - D_{recipient} \times C1$ 2. 从 P_b 的 x 坐标恢复字节 $b = x / k$ ``
D. 与 Method 42 的关系
`` Method 42 ECDH 密钥派生 (现有): → $S_{node} = ECDH(D_{node}, P_{node}).x$ → $aes_key = HKDF-SHA256(S_{node}, key_hash, \text{"bitfs-file-encryption"})$ → 用于 owner 自加密和 HTLC 购买
Koblitz 加密 (新增): → 用于第三方直接加密 (不需要 owner 参与) → 用于 key capsule 的加密传输 → 用于 Token 系统中的密钥交换
两者共存, 不互斥: - 单文件加密: Method 42 ECDH ($D_{node} + HKDF$ 确定性派生) - 密钥传输: Koblitz 加密 (发给特定接收方) - 内容加密: 始终 AES-256-GCM (对称加密) ``

六-B、Rabin 签名详细设计

本节补充 Rabin 签名在 Bitcoin Script 内的验证算法。

A. Rabin 签名算法

密钥生成：

1. 选择两个大素数 p, q (各 ≥ 1024 bits)
要求: $p \equiv 3 \pmod{4}, q \equiv 3 \pmod{4}$
2. 公钥: $n = p \times q$
3. 私钥: (p, q)

签名 (需要私钥):

1. 消息摘要: $H = \text{SHA256}(\text{message} || U)$
其中 U 是随机 padding, 使 H 成为 $\text{mod } n$ 的二次剩余
2. 计算签名: $S = H^{\{(n-p-q+5)/8\}} \text{mod } n$
(利用 $p \equiv q \equiv 3 \pmod{4}$ 的快速计算)
3. 签名输出: (S, U)

验证 (仅需公钥 n):

1. 重新计算: $H = \text{SHA256}(\text{message} || U)$
2. 验证: $S^2 \text{mod } n == H$
→ 无需分解 n , 仅需一次模乘和比较

B. Bitcoin Script 内验证

Rabin 签名验证可在 BSV Script 中实现 (不需要额外操作码):

Script (验证逻辑):

```
<S> <U> <message> <n>
OP_DUP                      → n n
<message> <U> OP_CAT OP_SHA256 → H
OP_SWAP                     → n H
<S> OP_DUP OP_MUL           → S2 (模乘)
OP_SWAP OP_MOD              → S2 mod n
OP_EQUAL                    → S2 mod n == H ?
```

优势:

- ECDSA 签名在 Script 中只能通过 OP_CHECKSIG 验证特定格式
- Rabin 签名可验证任意消息 (不限于交易签名)
- 适合: 第三方数据认证, Oracle 签名, 内容完整性证明

C. 在 BitFS 中的应用

内容认证：

1. 作者使用 Rabin 私钥 (p, q) 签名文件内容
2. 签名 (S, U) 和公钥 n 存储在 Metanet payload:
rabin_signature = encode(S, U)
rabin_pubkey = n
3. 任何人可验证: $S^2 \bmod n == \text{SHA256}(\text{content} || U)$

分片完整性：

1. 每个 chunk 单独签名
2. 验证：每个 chunk 的 Rabin 签名 → 确保内容未被篡改
3. 重组后验证: recombination_hash == H(chunk0||chunk1||...)

Script 内验证场景：

- 原子交换中要求内容真实性证明
- 数据交易的 locking script 可嵌入 Rabin 验证
- 第三方 Oracle 签名的外部数据源

七-B、CLTV 区块高度权限详细设计

八-B、内容分片与重组详细设计

本节补充大文件分片上链和重组的算法细节。

A. 分片策略

`` 分片触发: - 仅适用于链上存储模式
(onchain=true) - 链下模式不需要分片
(daemon 直接服务任意大小文件) - 阈值: 文件
大小 > BSV 交易大小上限 (当前 ~4GB, 但实际
推荐 < 10MB/tx)

分片算法: 1. chunk_size = 配置值 (默认 1MB)
2. 加密后的密文按 chunk_size 切分 3. 每个
chunk 独立发布为一笔数据交易
(BuildDataTransaction) 4. 元数据记录: -
total_chunks: 总分片数 - chunk_index: 当前
分片索引 (0-based) - content_txids: 所有分片
的 TxID 列表 (有序) - recombination_hash:
SHA256(chunk_0 || chunk_1 || ... ||
chunk_{N-1}) ``

B. 分片上传流程

七-B、CLTV 区块高度权限详细设计
<pre> ``` 上传 (bitfs put -onchain 大文件): 1. plaintext → key_hash = SHA256(SHA256(plaintext)) 2. 加密: ciphertext = AES-256-GCM(plaintext, sym_key) 3. 分片: chunks[] = split(ciphertext, chunk_size) 4. 计算: recombination_hash = SHA256(chunks[0] chunks[1] ...) 5. 对 每个 chunk[i]: Tx_data_i = BuildDataTransaction(chunk[i]) → Output 0: <chunk[i]> OP_DROP 6. 创建 Metanet 节 点交易: Tx_meta: OP_RETURN { type=FILE, onchain=true, content_txids=[Tx_data_0.TxID, Tx_data_1.TxID, ...], total_chunks=len(chunks), recombination_hash=recombination_hash, key_hash=key_hash, file_size=len(plaintext), ... } ``` </pre>
C. 重组下载流程
<pre> ``` 下载 (bget -onchain 或自动检测): 1. 获取元数据: GET /meta → payload 2. 检 查: payload.onchain == true 3. 对每个 content_txid: chunk[i] = 从区块链获取交易 → 提取 Output 0 的 OP_DROP 数据 4. 重组: ciphertext = chunks[0] chunks[1] ... 5. 验证: SHA256(ciphertext) == recombination_hash 6. 解密: plaintext = AES-256-GCM.Open(ciphertext, sym_key) 7. 验证: SHA256(SHA256(plaintext)) == key_hash ``` </pre>
D. 数据压缩
<pre> ``` 压缩流程 (加密前): 1. 检查 payload.compression 字段 2. 若 compression != NONE: compressed = </pre>

七-B、CLTV 区块高度权限详细设计

compress(plaintext, compression_scheme)
3. 加密: ciphertext = AES-256-GCM(compressed, sym_key) 4. key_hash = SHA256(SHA256(plaintext)) ← 注: 基于原始明文, 非压缩后

解压流程 (解密后): 1. 解密: compressed = AES-256-GCM.Open(ciphertext, sym_key)
2. 检查 compression 字段 3. 若 compression != NONE: plaintext = decompress(compressed, compression_scheme)

支持的压缩方案: NONE = 0 (默认, 不压缩) LZW = 1 GZIP = 2 ZSTD = 3 ``

九-B、CLI 命令详细参考

本节为所有 CLI 命令提供完整的参考手册, 与 `src/cmd/` 目录下的实现一致。

A. 只读工具 (b* 系列)

所有 b* 工具共享以下通用 flags:

<code>--json</code>	输出为 JSON 格式 (默认: false)
<code>--no-cache</code>	禁用本地缓存 (默认: false)
<code>--timeout N</code>	请求超时秒数 (默认: 30)
<code>--offline</code>	强制只用本地缓存数据 (默认: false)

bls - 列目录

用法: `bls [flags] bitfs://domain|pubkey/path`

Flags:

<code>-l, --long</code>	详细列表格式 (显示 type, name, size, date, access, hash)
<code>--keyword STR</code>	按关键词过滤文件名 (大小写不敏感)
<code>--json</code>	JSON 输出
<code>--no-cache</code>	禁用缓存
<code>--timeout N</code>	超时秒数 (默认 30)
<code>--offline</code>	仅用缓存

输出格式:

普通: name size date [access]
详细(-l): type name size date access hash
JSON: [{ "name": "...", "type": "...", "size": N, ... }]

退出码:

0 = 成功
1 = 一般错误 (解析失败, daemon 不可达)

bstat - 节点元数据

用法: bstat [flags] bitfs://domain|pubkey/path

Flags:

--versions 显示版本历史
--json JSON 输出
--no-cache 禁用缓存
--timeout N 超时秒数 (默认 30)
--offline 仅用缓存

输出格式:

普通:
File: readme.txt
Type: file
Hash: 3a7bd3e2...
Size: 4.2 KB
Owner: 02a1b2c3...
TxID: abc123...
Time: 2026-02-14 10:30:00 UTC
Access: free
MIME: text/plain
Desc: ...
JSON: 完整 NodeMetadata 对象

退出码:

0 = 成功
1 = 一般错误

bcats - 输出文件内容到 stdout

用法: bcats [flags] bitfs://domain|pubkey/path

Flags:

--buy 购买付费内容后输出
--json 输出元数据而非内容 (JSON 格式)
--no-cache 禁用缓存
--timeout N 超时秒数 (默认 30)
--offline 仅用缓存

行为:

- 免费文件: 自动解密 (D_node=1, FREE 模式 HKDF) 后输出到 stdout
- 付费文件 (无 --buy): 报错, 提示使用 --buy
- 付费文件 (有 --buy): 执行购买流程 → 解密 → 输出

退出码:

- 0 = 成功
- 1 = 一般错误

bget - 下载文件到本地

用法: bget [flags] bitfs://domain|pubkey/path

Flags:

- o, --output PATH 输出文件路径 (默认: URI 最后一段作为文件名)
- version N 下载特定版本 (默认: 0 = 最新)
- buy 购买付费内容
- json JSON 输出下载结果
- no-cache 禁用缓存
- timeout N 超时秒数 (默认 30)
- offline 仅用缓存

完整购买流程 (--buy):

1. 解析 URI → 连接 daemon endpoint
2. GET /meta → 获取元数据, 确认 access=PAID
3. 检查 key cache (~/.bitfs/cache/keys/{txid}.json)
 - 命中: 直接下载解密
 - 未命中: 继续购买流程
4. 生成临时 buyer 密钥对 (ec.NewPrivateKey)
5. POST /handshake → 建立 session
6. GET /buy/{txid}?buyer_pubkey → 获取定价 + capsule_hash
7. 本地构建 HTLC 脚本 (timeout=144 blocks)
8. POST /buy/{txid} {buyer_pubkey, htlc_tx} → 获取 capsule
9. 验证 SHA256(capsule) == capsule_hash
10. RecoverAESKey(capsule, D_buyer, P_seller, key_hash) → aes_key
11. 缓存 aes_key → ~/.bitfs/cache/keys/{txid}.json
12. GET /data/{hash} → 下载加密数据
13. AES-256-GCM 解密 → 写入输出文件

输出:

- 普通: "Downloaded [and decrypted] filename (size)"
- 购买: 显示 File, Price 信息

退出码:

- 0 = 成功
- 1 = 一般错误 (含 "content is paid; use --buy to purchase access")

btree – 递归目录树

用法: `btree [flags] bitfs://domain|pubkey/path`

Flags:

<code>--json</code>	JSON 输出
<code>--no-cache</code>	禁用缓存
<code>--timeout N</code>	超时秒数 (默认 30)
<code>--offline</code>	仅用缓存

输出格式:

普通: Unicode box-drawing 字符 (|—└─┐)

`example.com/`

```
|— docs/
|   |— readme.txt
|   └─ images/
|       └─ logo.png
└─ LICENSE
```

JSON: 嵌套 `TreeNode` 对象

退出码:

0 = 成功
1 = 一般错误

B. Owner 命令 (bitfs 子命令)

文件操作

`bitfs put <local> <remote>`

上传本地文件到远程路径

Flags:

<code>--encrypt</code>	加密上传 (private 模式, <code>encrypted=true</code>)
<code>--keyword "tags"</code>	空格分隔的关键词
<code>--description "..."</code>	文件描述
<code>--mime "type"</code>	MIME 类型覆盖 (默认: <code>application/octet-stream</code>)

交易: 新建 = 2 笔 (`CreateChild` + `SelfUpdate parent`)

更新 = 1 笔 (`SelfUpdate file`)

`bitfs mkdir <path>`

创建目录

交易: 2 笔 (`CreateChild DIR` + `SelfUpdate parent`)

`bitfs rm <path>`

删除文件 (仅移除父目录的 `ChildEntry`)

交易: 1 笔 (`SelfUpdate parent`)

注: 不发 `DELETE` 交易, 因硬链接可能引用同一 `P_node`

`bitfs rmdir <path>`

删除空目录

前置条件：目录必须为空（children 列表为空）

交易：2 笔（SelfUpdate parent + SelfUpdate dir 设 op=DELETE）

bitfs mv <src> <dst>

移动或重命名

同目录：1 笔（SelfUpdate parent, 修改 ChildEntry.Name）

跨目录：4 笔：

Tx1: CreateChild（目标新节点）

Tx2: SelfUpdate（源节点 → LINK SOFT 重定向）

Tx3: SelfUpdate（目标父目录，添加 ChildEntry）

Tx4: SelfUpdate（源父目录，标记 ChildEntry 为 LINK 类型）

bitfs cp <src> <dst>

复制（真复制：解密源 → 重新加密 → 创建新节点）

交易：2 笔（CreateChild + SelfUpdate parent）

注：新 P_node, 新 HD 路径, 新 file_index, 新 key_hash

bitfs link <target> <name>

硬链接（默认）

限制：仅文件，禁止目录硬链接（防止环路）

限制：仅本 Vault 内

交易：1 笔（SelfUpdate parent, 添加 ChildEntry 复用目标 P_node）

注：消耗 next_child_index 但不创建新 HD 密钥

bitfs link -s <target> <name>

本地软链接（创建 LINK 节点, link_type=SOFT）

link_target = 目标的 P_node（33 bytes 压缩公钥）

交易：2 笔（CreateChild LINK + SelfUpdate parent）

bitfs link -s <domain/path> <name>

远程软链接（创建 LINK 节点, link_type=SOFT_REMOTE）

link_target = "domain/path"（字符串）

交易：2 笔（CreateChild LINK + SelfUpdate parent）

加密管理

bitfs encrypt <path>

公开 → 私有（重新加密：D_node=1 FREE → D_node=BIP32 私钥 PRIVATE, HKDF 重新派生 aes_key）

交易：1 笔（SelfUpdate FILE, 更新 key_hash + access=PRIVATE）

操作：ReEncrypt(ciphertext, D_node, P_node, key_hash, file_index, FREE, PRIVATE)

bitfs decrypt <path>

私有 → 公开（重新加密：D_node=BIP32 私钥 PRIVATE → D_node=1 FREE, HKDF 重新派生 aes_key）

交易：1 笔（SelfUpdate FILE, 更新 key_hash + access=FREE）

操作：ReEncrypt(ciphertext, D_node, P_node, key_hash, file_index, PRIVATE, FREE)

交易

```
bitfs sell <path> --price <sat/KB>
标价出售（更新 Metanet 元数据）
Flags:
  --price N          单价 satoshis/KB（必填）
  --recursive        递归标价整个目录
交易：1 笔（SelfUpdate，设置 price_per_kb + access=PAID）

bitfs sales [path]
查看销售历史
```

发布

```
bitfs publish <domain> [path]
绑定域名到指定路径（默认 /）
操作：
  1. 引导用户配置 DNS TXT（_bitfs_pubkey）和 SRV（_bitfs._tcp）记录
  2. 更新 Metanet payload 中的 domain 字段
  3. 双向验证：DNS → Metanet + Metanet → DNS

bitfs unpublish <domain>
解除域名绑定

bitfs publish
列出所有绑定关系
```

钱包

```
bitfs wallet init
创建新 HD 钱包
流程：
  1. 选择助记词长度（12/24 words）
  2. 可选 passphrase
  3. 生成 BIP39 助记词
  4. 派生 master key（BIP39 seed）
  5. AES-256-GCM 加密 seed，存储到 wallet.db
输出：助记词（用户必须备份）

bitfs wallet restore
用助记词恢复 HD 密钥
注：仅恢复密钥树，tx 数据需从备份恢复

bitfs wallet info
显示余额、UTXO 数量、下一个收款地址
```



```
bitfs wallet fund
显示充值地址 (m/44'/236'/0'/0/{next_receive_index})
```

Vault

```
bitfs vault create <name>
  创建新 Vault (分配新 BIP44 account)

bitfs vault list
  列出所有 Vault

bitfs vault use <name>
  切换当前 Vault

bitfs vault info [name]
  显示 Vault 详情

bitfs vault rename <old> <new>
  重命名 Vault

bitfs vault delete <name>
  删除 Vault (标记删除)
```

Daemon

```
bitfs daemon start [-d]
  启动 HTTP daemon (-d 后台运行)

bitfs daemon stop
  停止 daemon

bitfs daemon status
  显示 daemon 状态

bitfs daemon config
  显示当前 daemon 配置
```

其他

```
bitfs init
  初始化向导: 创建钱包 → 备份助记词 → 创建首个 Vault → 显示充值地址

bitfs fsck
  文件系统一致性检查
```

```
bitfs shell
进入 FTP 风格交互 Shell（见第十节）
```

C. Shell 交互模式命令参考

Prompt 格式

```
bitfs /<当前远程路径>
```

例: `bitfs /docs>`, `bitfs />` (根目录)

Tab 补全行为

- 命令名补全: 输入部分命令名后 Tab 补全
- 路径补全: 远程路径和本地路径均支持 Tab 补全

完整命令参数表

分类	命令	参数	说明
远程导航	ls [path]	可选路径	列目录 (默认当前目录)
	cd <path>	必填路径	切换远程目录
	pwd	无	显示远程当前路径
	tree [path] [-d N]	可选路径, -d 深度限制	树形显示
	stat <path>	必填路径	节点详细信息
本地导航	lcd <path>	必填路径	切换本地目录
	lpwd	无	显示本地当前路径
	lls [path]	可选路径	列本地文件
传输	get <remote> [local]	远程路径, 可选本地路径	下载
	mget <pattern>	glob 模式	批量下载
	put <local> [remote]	本地路径, 可选远程路径	上传
	mput <pattern>	glob 模式	批量上传
	put -encrypt <local> [remote]	同 put + 加密	加密上传
远程操作	cat <file>	必填路径	输出内容 (自动解密)

分类	命令	参数	说明
	cp <src> <dst>	源和目标路径	复制
	mv <src> <dst>	源和目标路径	移动/重命名
	rm <path>	必填路径	删除文件
	mkdir <path>	必填路径	创建目录
	rmdir <path>	必填路径	删除空目录
	link <target> <name>	目标和链接名	硬链接
	link -s <target> <name>	同上	软链接
加密	encrypt <path>	必填路径	公开→私有
	decrypt <path>	必填路径	私有→公开
交易	sell <path> -price N	路径 + 价格	标价出售
	sales [path]	可选路径	销售历史
钱包	balance	无	查看余额
	fund	无	显示充值地址
发布	publish <domain> [path]	域名 + 可选路径	绑定域名
	unpublish <domain>	域名	解除绑定
	publish	无	查看绑定关系
Vault	vault list	无	列出 Vault
	vault use <name>	Vault 名称	切换 Vault
会话	! <cmd>	Shell 命令	执行本地命令
	help	无	帮助
	history	无	命令历史
	exit / quit / bye	无	退出

十三-B、节点间通信协议详细设计
十一-B、Token 系统详细设计
本节补充 Hash Chain 批量预购令牌系统的算法细节。
A. Hash Chain 原理
`` 令牌生成 (Buyer): 1. 选择种子: $Y = \text{random}(32 \text{ bytes})$ 2. 生成令牌链: $T_0 = Y, T_i = \text{SHA256}(T_{i-1})$ ($i=1..N$) 3. 锚点: $T_N = \text{SHA256}^N(Y)$ (最终哈希)
链结构: $T_0 = Y \leftarrow$ 最后使用 (第 N 次购买) $T_1 = \text{SHA256}(Y) \leftarrow$ 第 $N-1$ 次购买 $T_2 = \text{SHA256}(\text{SHA256}(Y)) \leftarrow$ 第 $N-2$ 次购买 $\dots T_{N-1} = \text{SHA256}^{N-1}(Y) \leftarrow$ 第 1 次购买 $T_N = \text{SHA256}^N(Y) \leftarrow$ 锚点 (公开, 写入合约)
令牌验证 (Seller): 收到 T_i 时: 1. 验证: $\text{SHA256}^{N-i}(T_i) == T_N$ (锚点) 2. 验证: T_i 未被使用过 3. 通过 $\rightarrow$ 交付解密密钥 ``
B. 预购流程
`` 阶段 1: 建立 (Setup) Buyer: 1. 确定购买次数 N (如: 100 次) 2. 生成 Hash Chain: $Y \rightarrow T_1 \rightarrow \dots \rightarrow T_N$ 3. 构建预购交易: - 锁定金额: $N \times \text{price_per_file}$ - 合约包含: T_N (锚点), N (总次数), P_{seller}

十三-B、节点间通信协议详细设计
链上合约: OP_SHA256 OP_EQUAL ← 验证令牌 OP_IF OP_CHECKSIG ← Seller 签名 领取 OP_ELSE OP_CHECKSEQUENCEVERIFY OP_DROP OP_CHECKSIG ← Buyer 超时退款 OP_ENDIF
阶段 2: 兑换 (Redeem) 第 k 次购买 (k=1..N): Buyer → Seller: { token: T_{N-k}, file_txid: <要购买的文件> } Seller: 1. 验证 SHA256^k(T_{N-k}) == T_N 2. 验证 T_{N-k} 未使用 3. 标记 T_{N-k} 已使用 4. 调用 CreateKeyCapsule → capsule 5. 返回 { capsule, capsule_hash } Seller 链上: 提交 T_{N-k} 领取对应金额 (1/ N 的锁定金额)
阶段 3: 结算 (Settle) - 所有令 牌使用完毕: Seller 已领取全部 金额 - 部分使用: 超时后 Buyer 可退回剩余 ``
C. 令牌验证算法
``go // VerifyToken 验证令牌 是否属于指定 Hash Chain func VerifyToken(token []byte, anchor []byte, depth int) bool { current := token for i := 0; i < depth; i++ { h := sha256.Sum256(current) current = h[:] } return

十三-B、节点间通信协议详细设计
bytes.Equal(current, anchor) }
<pre>// 示例: // 第 1 次购买: VerifyToken(T_{N-1}, T_N, 1) → SHA256(T_{N-1}) == T_N ✓ // 第 2 次购买: VerifyToken(T_{N-2}, T_N, 2) → SHA256(SHA256(T_{N-2})) == T_N ✓ // 第 k 次购买: VerifyToken(T_{N-k}, T_N, k) → SHA256^k(T_{N-k}) == T_N ✓</pre>
<pre>// Seller 维护已使用计数器, 确定 depth 参数 type TokenState struct { Anchor []byte // T_N (锚点哈希) TotalUses int // N (总可用次数) UsedCount int // 已使用次数 }</pre>
<pre>// 验证时: k = state.UsedCount + 1 // Buyer 应按序揭示: T_{N-1}, T_{N-2}, ..., T_0 // 跳序揭示 (如直接揭示 T_{N-3}) 会浪费中间 token ``</pre>
D. 安全性分析
<pre>`` 防重放: - Seller 维护已使用 令牌集合 - 每个 T_i 只能使用一次</pre>
<pre>防伪造: - 攻击者知道 T_N (公开) 和已揭示的 T_{N-1}, T_{N-2}, ... - 要伪造 T_{N-k-1}</pre>

十三-B、节点间通信协议详细设计

需要找到 SHA256 的原像 → 计算不可行

防提前揭示: - Buyer 按逆序揭示: $T_{\{N-1\}}, T_{\{N-2\}}, \dots, T_0$ - 知道 $T_{\{N-k\}}$ 无法推导出 $T_{\{N-k-1\}}$ (单向哈希)

退款保证: - 超时后 Buyer 可赎回未使用的锁定金额 - HTLC 风格的
OP_CHECKSEQUENCEVERIFY 保证 ``

十二-B、Git Remote Helper 详细设计

本节补充 `git-remote-bitfs` 的内部架构、协议细节、错误处理和边界情况。

A. 二进制与内部架构

```
implementation/src/cmd/git-remote-bitfs/
├─ main.go           ← 入口: 解析 URL, 分发 capabilities/list/import/export
├─ protocol.go       ← git remote helper stdin/stdout 协议解析
├─ exporter.go       ← export: fast-export 流 → packfile → 加密 → 上传
├─ importer.go       ← import: 下载 → 解密 → fast-import 流
├─ refs.go           ← .git-refs 读写 (JSON)
├─ transport.go      ← 与 BitFS daemon/本地的通信层
└─ config.go         ← git config 读取 (wallet, vault, autoConfirm)
```

依赖核心库: - `internal/method42` — 加密/解密, ECDH 握手, key capsule - `internal/cli` — HTTP client (与 daemon 通信) - `internal/metanet` — Metanet 节点操作 (refs 更新) - `internal/storage` — 内容寻址存储 (packfile 存取) - `internal/config` — 配置读取 - `os/exec` — 调用 git fast-import/fast-export/pack-objects

B. 临时工作目录

helper 执行过程中使用临时 bare repo:

```
~/bitfs/tmp/git-remote-<pid>/
├─ bare.git/          ← 临时 bare repo (fast-import 目标)
└─ packs/             ← 下载的解密 packfile 暂存
```

操作完成后自动清理。大仓库可通过 `git config bitfs.tmpDir` 指定位置。

C. 错误处理

场景	行为
daemon 不可达	stderr 输出错误, git 报 remote error
付费被拒绝 (用户取消)	abort import, git 报 “authentication failed”
HTLC 超时	自动退款, stderr 提示重试
push 权限不足	daemon 返回 403, git 报 “permission denied”
non-fast-forward	与标准 git 一致, 提示先 fetch
packfile 损坏 (解密后校验失败)	stderr 报错, 不导入
网络中断 (上传中)	已上传的 packfile 留在存储 (幂等), refs 未更新故无副作用

D. 边界情况

情况	处理
空仓库 clone	.git-refs 不存在 → 返回空 refs 列表 → git 创建空 repo
submodule 指向 bitfs://	天然支持, git 递归调用 git-remote-bitfs
超大 packfile (>100MB)	警告但不阻止, BitFS 内容寻址存储无大小限制
shallow clone (-depth)	一期不支持, 返回完整历史
LFS 文件	不需要, BitFS 本身就是 LFS — 所有文件大小一视同仁

E. Repack (后续优化)

多次增量 push 后会积累大量小 packfile。一期不实现 repack, 多 packfile 不影响正确性, 只影响 clone 速度。后续提供:


```
bitfs git-repack /projects/myapp # 合并所有 packfile 为一个
```

十四-B、收益权表与 ISO 详细设计

本节补充收益权 UTXO 化、Registry Covenant、ISO 发行机制的算法细节。

A. 双层 Covenant 互锁机制

Share UTXO Covenant

验证规则：

1. OP_CHECKSIG: 持有者身份验证
2. OP_PUSH_TX: 获取当前花费交易
3. 检查 Input[0] 为 Registry UTXO
→ 通过 Registry 的 script hash 识别
4. 份额守恒: $\text{sum}(\text{output_shares}) == \text{input}$
5. node_id 不变: 输出绑定同一 Metanet 节点

允许操作：

- 转让: 1 input → 1 output (不同 holder)
- 拆分: 1 input → N outputs (sum 守恒)
- 合并: N inputs → 1 output (sum 守恒)

互锁: 同一笔交易

Registry UTXO Covenant

状态数据 (OP_PUSH_DATA 编码):

node_id: bytes32 (Metanet 节点 ID)
total: uint64 (总份额)
num_entries: uint32 (股东数量)
entries[]: (addr:bytes20, share:uint64) []

MODE_TRANSFER (份额转让):

1. 检查至少一个 Share UTXO 在 inputs 中
2. 验证旧 entry 与 Share input 匹配
3. 验证新 entry 与 Share outputs 匹配
4. $\text{sum}(\text{new_entries.share}) == \text{sum}(\text{old})$
5. 输出新 Registry UTXO

MODE_DISTRIBUTE (收益分配):

1. 读取 entries []

- | |
|---|
| 2. 验证 <code>Output[i+1].value >= payment * entries[i].share / total</code> |
| 3. 验证 <code>Output[i+1].script == P2PKH(entries[i].addr)</code> |
| 4. 输出新 Registry UTXO (状态不变) |

B. Registry UTXO 状态编码

Registry 状态序列化格式 (嵌入 OP_PUSH_DATA):

Offset	Size	Field
0	32	node_id (Metanet 节点的 SHA256(P_node TxID))
32	8	total_shares (uint64, big-endian)
40	4	num_entries (uint32, big-endian)
44	24×N	entries: [0..19] address (20-byte P2PKH hash) [20..27] share (uint64, big-endian)
44+24N	1	mode_flags: bit 0: ISO active (ISO Pool 存在) bit 1: locked (禁止转让, 创作者可设)

总大小: $45 + 24 \times \text{num_entries}$ bytes

C. Share UTXO 编码

Share UTXO 锁定脚本 (ScriptPubKey):

```
<share_data> OP_DROP          // 份额数据 (node_id + share_amount)
OP_DUP OP_HASH160 <holder_pkh> OP_EQUALVERIFY // 持有者身份
// Covenant 验证 (OP_PUSH_TX / OP_CHECKSIGPREIMAGE):
// 1. 从 sighash preimage 提取当前交易的输出列表
// 2. 验证输出[0] 包含 Registry UTXO (Script hash 匹配)
// 3. 从输出[0] 的 Script 中提取 share_data
// 4. 验证: 输入 share_total == 输出 share_total (份额守恒)
// 5. 验证: 新 holder_pkh 是有效的 P2PKH 地址
OP_CHECKSIG
```

share_data 格式:

Offset	Size	Field
0	32	node_id
32	8	share_amount (uint64, big-endian)

总大小: 40 bytes

D. ISO Pool Covenant 算法

ISO Pool UTX0 状态:

```
node_id:          bytes32 (绑定的 Metanet 节点)
remaining_shares: uint64  (剩余可购份额)
price_per_share:  uint64  (每份价格, satoshi)
creator_addr:     bytes20 (创作者收款地址)
```

ISO Pool Covenant 验证逻辑:

MODE_BUY (任何人可触发):

Input:

buy_amount: uint64 (购买份额数)

验证:

1. buy_amount > 0
2. buy_amount <= remaining_shares
3. Output 包含 Registry UTX0 (更新: 加入 buyer entry)
4. Output 包含 Share UTX0 (buyer_addr, buy_amount)
5. Output 包含 P2PKH → creator_addr, value >= buy_amount × price
6. 如果 remaining - buy_amount > 0:
Output 包含新 ISO Pool UTX0 (remaining - buy_amount)
7. Output 份额总和 == Input 份额总和

MODE_CLOSE (仅创作者):

验证:

1. Input 包含创作者签名
2. Output 包含 Registry UTX0 (ISO_POOL entry 移除或转给创作者)
3. 如选择回收: 创作者 share 增加 remaining_shares
4. 如选择销毁: total_shares 减少 remaining_shares

E. 购买分账算法

```
// DistributeRevenue 计算每个股东的收益分配
func DistributeRevenue(totalPayment uint64, entries []RevShareEntry, totalShares uint64) []Distribution {
    distributions := make([]Distribution, len(entries))
    var distributed uint64

    for i, entry := range entries {
        if i == len(entries)-1 {
            // 最后一个股东获得剩余 (避免精度损失)
            distributions[i] = Distribution{
                Address: entry.Address,
                Amount: totalPayment - distributed,
            }
        } else {
            amount := totalPayment * entry.Share / totalShares
            distributions[i] = Distribution{
```

```

        Address: entry.Address,
        Amount: amount,
    }
    distributed += amount
}
}
return distributions
}

// 示例:
// totalPayment = 10000 sat
// entries = [(A, 3000), (B, 2000), (C, 5000)]
// totalShares = 10000
//
// A:  $10000 \times 3000 / 10000 = 3000$  sat
// B:  $10000 \times 2000 / 10000 = 2000$  sat
// C:  $10000 - 3000 - 2000 = 5000$  sat (最后一个取剩余)

```

F. 份额拆分与合并验证

拆分验证 ($1 \rightarrow N$):

```

Input:  Share UTX0 (node_id, amount_in)
Output: Share UTX0[0] (node_id, amount_0)
        Share UTX0[1] (node_id, amount_1)
        ...
        Share UTX0[N-1] (node_id, amount_{N-1})

```

验证规则:

1. 所有 output 的 node_id == input 的 node_id
2. $\text{sum}(\text{amount}_0 \dots \text{amount}_{\{N-1\}}) == \text{amount}_{\text{in}}$
3. 每个 $\text{amount}_i > 0$
4. Registry UTX0 同步更新 (互锁)

合并验证 ($N \rightarrow 1$):

```

Input:  Share UTX0[0] (node_id, amount_0) ← 同一 holder
        Share UTX0[1] (node_id, amount_1) ← 同一 holder
        ...
Output: Share UTX0 (node_id, sum(amounts))

```

验证规则:

1. 所有 input 的 node_id 相同
2. 所有 input 的 holder 相同 (同一签名者)
3. $\text{output amount} == \text{sum(input amounts)}$
4. Registry UTX0 同步更新

原子交换 (份额买卖):

```

Input:  Registry UTX0
        Share UTX0 (seller, amount) ← seller 签名
        Payment UTX0 (buyer)         ← buyer 签名

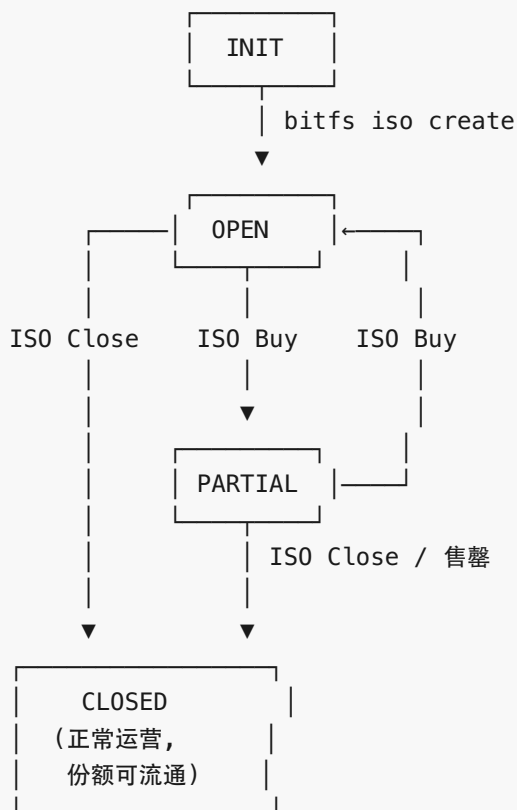
```

Output: Registry UTX0 (更新)
Share UTX0 (buyer, amount) → 1 sat
P2PKH → seller → payment

验证规则:

1. Share Covenant: Registry 在 Input[0]
2. Registry Covenant: Share 在 inputs
3. 份额守恒
4. 支付金额正确 (双方签名同意)

G. ISO 状态机



H. 安全性分析

Covenant 互锁保证:

- Registry 和 Share UTX0 不可能单独修改
- 攻击者无法:
 1. 伪造份额: 无法创建新 Share UTX0 不更新 Registry
 2. 篡改注册表: 无法更新 Registry 不花费 Share UTX0
 3. 双花份额: BSV UTX0 模型天然防止
 4. 窃取收益: 分配金额由 Covenant 脚本验证

ISO Pool 安全:

- 超卖: Covenant 验证 $\text{buy_amount} \leq \text{remaining}$
- 低价: Covenant 验证 $\text{payment} \geq \text{buy_amount} \times \text{price}$
- 只有创作者可关闭 ISO (签名验证)

份额转让安全:

- 只有当前持有者可转让 (P2PKH 签名)
- 份额守恒: Covenant 验证 $\text{sum}(\text{output}) == \text{sum}(\text{input})$
- `node_id` 不可变: 份额始终绑定原始文件

精度安全:

- 分配算法最后一个股东取剩余, 避免整除精度损失
- 单笔分红支付最低金额 = 股东数量 $\times$ 546 satoshis (BSV P2PKH dust limit = 546 sat)

十五-B、目录树购买与 BIP32 访问控制详细设计

本节补充基于 BIP32 非硬化派生的目录树级购买机制和访问控制算法。

A. ECDH 传递性证明

给定:

```
D_parent      = 父节点私钥
P_parent      = D_parent × G
chaincode     = 父节点 chain code
index         = 子节点编号
P_buyer       = 买方公钥
D_buyer       = 买方私钥
```

BIP32 非硬化派生:

```
offset = HMAC-SHA512(chaincode, P_parent || index)[:32]
D_child = D_parent + offset (mod n)
P_child = P_parent + offset × G
```

ECDH 共享密钥:

```
S_parent = D_parent × P_buyer = D_buyer × P_parent    (双方可独立计算)
S_child  = D_child × P_buyer
          = (D_parent + offset) × P_buyer
          = D_parent × P_buyer + offset × P_buyer
          = S_parent + offset × P_buyer
```

Buyer 推导 (不需要 `D_child`):

```
已知: S_parent (从 capsule 获得), chaincode (从 xpub 获得), P_buyer (自己的)
计算: offset = HMAC-SHA512(chaincode, P_parent || index)[:32]
计算: S_child = S_parent + offset × P_buyer ✓
```

递归:

```
S_grandchild = S_child + offset_2 × P_buyer
```

任意深度的非硬化子节点都可推导

B. 调整后的加密密钥派生

旧 Method 42:

```
Df(0) = SHA256(D_node || key_hash || file_index)
S_k = ECDH(Df(0), P_buyer)
aes_key = KDF(S_k)
```

问题: SHA256 打断 BIP32 代数关系, 无法从父 ECDH 推导子 ECDH

新方案:

```
S_node = ECDH(D_node, P_buyer)      ← BIP32 密钥, 保留代数关系
aes_key = KDF(S_node, key_hash)      ← key_hash 在 KDF 阶段提供唯一性
```

其中:

```
S_node = D_node × P_buyer           (卖方计算)
        = D_buyer × P_node           (买方计算)
KDF(S, key_hash) = HKDF-SHA256(
    ikm = S.x,                       // ECDH 共享密钥的 x 坐标
    salt = key_hash,                  // SHA256(SHA256(plaintext))
    info = "bitfs-file-encryption",
    length = 32                       // AES-256 密钥
)
```

安全性对比

	旧方案 (Df(0))	新方案 (D_node + KDF)
内容唯一性	✓ (key_hash in Df)	✓ (key_hash in KDF)
买方唯一性	✓ (P_buyer in ECDH)	✓ (P_buyer in ECDH)
文件唯一性	✓ (D_node unique)	✓ (D_node unique)
BIP32 可推导	✗ (SHA256 打断)	✓ (D_node 直接 ECDH)
目录树购买	✗	✓
硬化/非硬化访问控制	✗	✓

C. 目录树购买算法

```
// BuyDirectory 购买整个目录树的非硬化子节点
func BuyDirectory(dirPath string, buyerPrivKey *ec.PrivateKey) error {
    // 1. 与 Seller 握手
    session := Handshake(dirPath, buyerPrivKey.PubKey())

    // 2. 获取目录的 xpub 和非硬化子节点列表
    dirInfo := session.GetDirectoryInfo(dirPath)
    xpub := dirInfo.Xpub      // P_dir + chaincode
    children := dirInfo.NonHardenedChildren
```

```

// 3. 计算总价
totalPrice := uint64(0)
for _, child := range children {
    totalPrice += child.PricePerKB * child.FileSize / 1024
}

// 4. 一笔 HTLC 支付
capsuleHash := dirInfo.CapsuleHash
htlcTx := CreateHTLC(totalPrice, capsuleHash, session.SellerPubKey)
Broadcast(htlcTx)

// 5. Seller 揭示 capsule (S_parent = ECDH(D_dir, P_buyer))
capsule := WaitForCapsuleReveal(htlcTx) // S_parent point

// 6. 本地派生所有子密钥并解密
for _, child := range children {
    sChild := DeriveChildECDH(capsule, xpub, child.Index, buyerPrivKey.PubKey())
    aesKey := HKDF(sChild.X(), child.KeyHash, "bitfs-file-encryption", 32)
    plaintext := AESGCMDecrypt(child.EncryptedContent, aesKey)
    Store(child.Path, plaintext)
}
return nil
}

// DeriveChildECDH 从父 ECDH 共享密钥推导子 ECDH
func DeriveChildECDH(
    sParent *ec.Point, // 父节点 ECDH: D_parent × P_buyer
    xpub *ExtendedPubKey, // 父节点 xpub
    childIndex uint32, // 子节点 index
    buyerPub *ec.PublicKey, // 买方公钥
) *ec.Point {
    // offset = HMAC-SHA512(chaincode, P_parent || index)[:32]
    data := append(xpub.PubKey.Compressed(), uint32ToBytes(childIndex)...)
    hmacResult := HMAC_SHA512(xpub.ChainCode, data)
    offset := new(big.Int).SetBytes(hmacResult[:32])

    // S_child = S_parent + offset × P_buyer
    offsetPoint := ec.ScalarMul(buyerPub.Point(), offset)
    return ec.PointAdd(sParent, offsetPoint)
}

```

D. 硬化/非硬化访问控制矩阵

创建子节点时的派生选择：

bitfs put file.md /dir/	→ 非硬化（默认）
D_child = D_parent + offset	（offset 从 xpub 可计算）
目录购买者可解密	✓


```
bitfs put file.md /dir/ --exclusive → 硬化
D_child = HMAC(D_parent, index)    (需要 D_parent 私钥)
目录购买者无法解密                  x (需单独购买)
```

访问控制矩阵:

购买方式	非硬化子节点	硬化子节点
单文件 HTLC	✓	✓
Token 兑换	✓	✓
目录 xpub	✓	x
父目录 xpub	✓ (递归)	x

E. 递归目录树解锁

非硬化子目录也是 xpub 可推导的, 形成递归解锁:

```
/premium/                ← 购买此目录
├─ intro.md              (非硬化) ← 可解密 ✓
├─ part1/                (非硬化) ← 子目录也可推导
│   ├─ ch1.md            (非硬化) ← 递归可解密 ✓
│   ├─ ch2.md            (非硬化) ← 递归可解密 ✓
│   └─ answers.md        (硬化)   ← 排除 x
├─ part2/                (非硬化) ← 子目录也可推导
│   └─ ...                (非硬化) ← 递归可解密 ✓
└─ exam/                 (硬化)   ← 整个子目录排除 x
    └─ ...                ← 全部排除 x
```

算法:

1. 获取根目录 capsule (S_root)
2. BFS/DFS 遍历目录树
3. 遇到非硬化节点: 推导 S_child, 继续递归
4. 遇到硬化节点: 跳过 (无法推导)

F. 订阅与时效

目录购买 + CLTV = 限时订阅:

购买 capsule 的 HTLC 包含 CLTV:

```
IF (SHA256(preimage) == capsule_hash AND block_height < expiry AND Sig(seller))
ELSE IF (timeout AND Sig(buyer)) → 退款
```

Buyer 获得 capsule 后:

- 当前非硬化文件: 可解密 (capsule 已获得, 永久有效)
- 未来非硬化文件: xpub 派生可用, 但下载需要 Seller daemon 配合
- Seller daemon 在 block_height > expiry 后拒绝提供加密内容

- 已下载的文件仍可本地解密（密钥已在本地）

续订：

- 新 HTLC → 新 capsule（同一 xpub，但 Seller 可能更换了某些子节点）
- 或：Token 预购 N 个月的访问权

G. 安全性分析

ECDH 传递性安全：

- 知道 S_{parent} 和 $xpub$ → 可推导所有非硬化 S_{child}
- 知道 S_{parent} 和 $xpub$ → 不能推导 D_{parent} （ECDL 困难）
- 知道 S_{child} → 不能推导 S_{parent} （单向：加法容易，减法需知 $offset \times P_{buyer}$ ）

实际上 $offset$ 是公开的，所以 $S_{parent} = S_{child} - offset \times P_{buyer}$

→ 子节点的 capsule 泄露会暴露父目录 capsule！

****解决方案（设计决策 #82）**：**

- 默认硬化派生：ChildEntry.hardened 默认为 true，切断 parent↔child 代数关系
- 硬化派生： $D_{child} = \text{HMAC-SHA512}(\text{chaincode}, 0x00 || D_{parent} || \text{index})[:32]$
- 无法从 D_{child} 反推 D_{parent} ，因此子 capsule 无法推导父 capsule
- 非硬化为显式 opt-in：仅当 Owner 创建子节点时设置 hardened=false
- 适用场景：“子树购买” - 买家购买任意后代节点即获得子树根的访问权
- Owner 明确理解并接受此安全权衡
- 实际效果：大多数文件使用硬化派生（独立安全），仅标记为可子树购买的目录使用非硬化

硬化节点安全：

- 硬化派生： $D_{child} = \text{HMAC-SHA512}(\text{chaincode}, 0x00 || D_{parent} || \text{index})[:32]$
- 需要 D_{parent} 私钥才能计算 → Buyer 无法推导
- 即使 Buyer 有 $xpub + S_{parent}$ ，也无法计算硬化子节点的 ECDH

capsule 不可转让：

- $\text{capsule} = \text{ECDH}(D_{dir}, P_{buyer}) = D_{dir} \times P_{buyer}$
- 绑定到特定 P_{buyer}
- 其他人知道 capsule 也无法解密（需要 D_{buyer} 来配合 KDF）
- 但：如果 Buyer 分享 aes_key ，任何人都能解密 → 与所有 DRM 一样，无法防止密钥分享

目录购买范围：

- 非硬化：买了目录就永久可解密该目录下所有非硬化内容（包括未来新增）
- 硬化：永远排除，即使 Buyer 有目录 capsule
- 创作者通过选择硬化/非硬化来精确控制访问范围

十六-B、Paymail 集成详细设计

本节详细设计 Paymail (bsvalias) 协议与 BitFS 系统的集成，包括 URI 解析算法、Daemon Server 实现、自定义 BRFC 注册，以及与 Method 42 握手的桥接。

A. URI 解析算法

BitFS URI 扩展为三种模式, 遵循 RFC 3986 userinfo@host 语法:

```
// ResolveURI 解析 bitfs:// URI, 支持三种寻址模式
func ResolveURI(uri string) (*ResolveResult, error) {
    parsed := parseURI(uri) // scheme://authority/path
    authority := parsed.Authority

    if strings.Contains(authority, "@") {
        // Mode 1: Paymail — bitfs://alice@example.com/path
        parts := strings.SplitN(authority, "@", 2)
        alias, domain := parts[0], parts[1]
        return resolvePaymail(alias, domain, parsed.Path)
    }

    if isHexPubKey(authority) {
        // Mode 2: 裸公钥 — bitfs://02a1b2c3.../path
        pubkey, _ := hex.DecodeString(authority)
        return &ResolveResult{PNode: pubkey, Path: parsed.Path}, nil
    }

    // Mode 3: DNSLink — bitfs://example.com/path
    return resolveDNSLink(authority, parsed.Path)
}

func resolvePaymail(alias, domain, path string) (*ResolveResult, error) {
    // Step 1: SRV lookup (Paymail host discovery)
    srv := lookupSRV("_bssvalias", "_tcp", domain)

    // Step 2: Capability discovery
    capURL := fmt.Sprintf("https://%s:%d/.well-known/bssvalias", srv.Host, srv.Port)
    caps := httpGET(capURL)

    // Step 3: PKI — 获取公钥
    pkiURL := caps.GetTemplate("pki")
    pkiURL = strings.Replace(pkiURL, "{alias}", alias, 1)
    pkiURL = strings.Replace(pkiURL, "{domain.tld}", domain, 1)
    pki := httpGET(pkiURL)

    pubkey, _ := hex.DecodeString(pki.PubKey)

    return &ResolveResult{
        PNode:  pubkey,
        Endpoints: srvToEndpoints(srv),
        Domain:  domain,
        Alias:   alias,
        Path:    path,
    }, nil
}
```

```
}, nil  
}
```

B. Daemon Paymail Server

BitFS daemon 同时作为 Paymail server, 在现有 HTTP 路由中添加 Paymail endpoints。

路由注册

```
// 现有 BitFS 路由 (保留)  
mux.HandleFunc("/data/{hash}", handleData)  
mux.HandleFunc("/meta/{pnode}/{path}", handleMeta)  
mux.HandleFunc("/buy/{txid}", handleBuy)  
mux.HandleFunc("/handshake", handleHandshake)  
  
// Paymail 路由 (新增)  
mux.HandleFunc("/.well-known/bsvalias", handleCapabilities)  
mux.HandleFunc("/api/v1/pki/{paymail}", handlePKI)  
mux.HandleFunc("/api/v1/profile/{paymail}", handleProfile)  
mux.HandleFunc("/api/v1/verify/{paymail}/{pubkey}", handleVerifyPubKey)
```

Capabilities 响应

```
func handleCapabilities(w http.ResponseWriter, r *http.Request) {  
    host := r.Host  
    caps := map[string]interface{}{  
        "bsvalias": "1.0",  
        "capabilities": map[string]interface{}{  
            "pki":      fmt.Sprintf("https://%s/api/v1/pki/{alias}@{domain.tld}", host),  
            "f12f968c92d6": fmt.Sprintf("https://%s/api/v1/profile/{alias}@{domain.tld}", host),  
            "a9f510c16bde": fmt.Sprintf("https://%s/api/v1/verify/{alias}@{domain.tld}/{pubkey}", host),  
        },  
    }  
    json.NewEncoder(w).Encode(caps)  
}
```

PKI 响应

```
func handlePKI(w http.ResponseWriter, r *http.Request) {  
    alias, domain := parsePaymail(r.PathValue("paymail"))  
  
    // alias → vault 名称映射  
    vault, err := findVaultByAlias(alias)  
    if err != nil {  
        http.Error(w, "not found", 404)  
        return  
    }  
}
```

```

json.NewEncoder(w).Encode(map[string]string{
    "bsvalias": "1.0",
    "handle":  alias + "@" + domain,
    "pubkey":  hex.EncodeToString(vault.RootPubKey()),
})
}

```

Vault-Alias 映射

Daemon 配置中声明 alias → vault 映射:

```

# ~/.bitfs/config.yaml
paymail:
  enabled: true
  aliases:
    alice: personal    # alice@domain → personal vault
    photos: media      # photos@domain → media vault

```

无配置时, 默认 alias = vault 名称。

C. 与 Method 42 握手的桥接

Paymail PKI 提供的公钥可直接用于 Method 42 ECDH 握手:

握手流程 (Paymail 增强):

Client (Buyer)	Server (Seller)
-- Handshake Request ----->	
{ paymail: "bob@handcash.io",	
nonce_b: <32 bytes>,	
pubkey_b: <33 bytes> }	
Server 验证:	
1. PKI lookup bob@handcash.io	
→ 获取 P_bob	
2. 验证 pubkey_b == P_bob	
3. 验证 nonce_b 签名	
→ 身份确认: 对方确实是 bob@handcash.io	
<--- Handshake Response -----	
{ nonce_s, pubkey_s, hmac_s }	
双方计算:	
session_key = SHA256(ECDH_shared_x nonce_b nonce_s)	

Paymail 身份验证是可选增强 — 无 paymail 时降级为现有公钥直接验证。

D. 收益权分红地址解析

Revenue Share 分红时, 股东地址可通过 Paymail Address Resolution 获取:

分红流程:

1. Registry UTXO 中记录股东身份:
 - 方式 A: BSV address (现有, 静态)
 - 方式 B: Paymail handle (新增, 动态)
2. 分红时解析:

```
if 股东有 paymail:
    address = paymail.resolveAddress(handle) // 每次返回新 HD 地址
else:
    address = 股东注册时的 BSV address      // 静态地址
```
3. 优势:
 - 每次分红发到不同地址 → 增强隐私
 - 股东换钱包只需更新 Paymail 服务端
 - 无需更新链上 Registry

注意: 这要求分红操作在链下执行 (daemon 构建交易时解析 paymail), 然后广播。Registry Covenant 本身只验证金额分配正确, 不关心具体地址来源。

E. 自定义 BRFC 注册

BitFS 可注册自定义 BRFC capabilities, 使 Paymail 客户端发现 BitFS 特有功能:

BRFC ID 生成:

```
title   = "BitFS Browse"
author  = "BitFS"
version = "1.0"
id = SHA256d(title + author + version)[:12] // 截取前 6 字节 hex
```

潜在自定义 capabilities:

```
bitfs-browse: 浏览 Vault 目录结构
bitfs-buy:    发起 HTLC 购买
bitfs-sell:   查询售价信息
```

这些自定义 BRFC 使得未来的通用 Paymail 客户端也能与 BitFS 节点交互, 无需专用 BitFS 客户端。

F. 安全性分析

Paymail PKI 信任模型:

- Paymail 公钥由域名持有者的服务端返回
- 信任基础: DNS + HTTPS (与传统 Web 相同)

- 不如链上 Metanet 的密码学验证强（链上不可篡改）

缓解：

- Paymail 仅用于身份发现，不用于密钥派生
- 实际加密/签名仍使用链上 P_node
- Method 42 握手验证对方确实持有 D_node 私钥
- 即使 Paymail PKI 被篡改，ECDH 握手会失败 → 无法中间人攻击

DNS 安全：

- Paymail 依赖 DNS SRV (_bsvalias._tcp)
- 建议启用 DNSSEC
- go-paymail 库支持 DNSSEC 检查

Paymail 与 DNSLink 冲突：

- 两套 DNS 记录独立：_bsvalias._tcp vs _bitfs._tcp
- 可指向同一服务器，不冲突
- Paymail 多用户，DNSLink 单用户 → 互补而非替代

Metanet Chain 详细设计已移至独立文档: [../metanet/3-DetailedDesign.zh.md](https://metanet/3-DetailedDesign.zh.md)